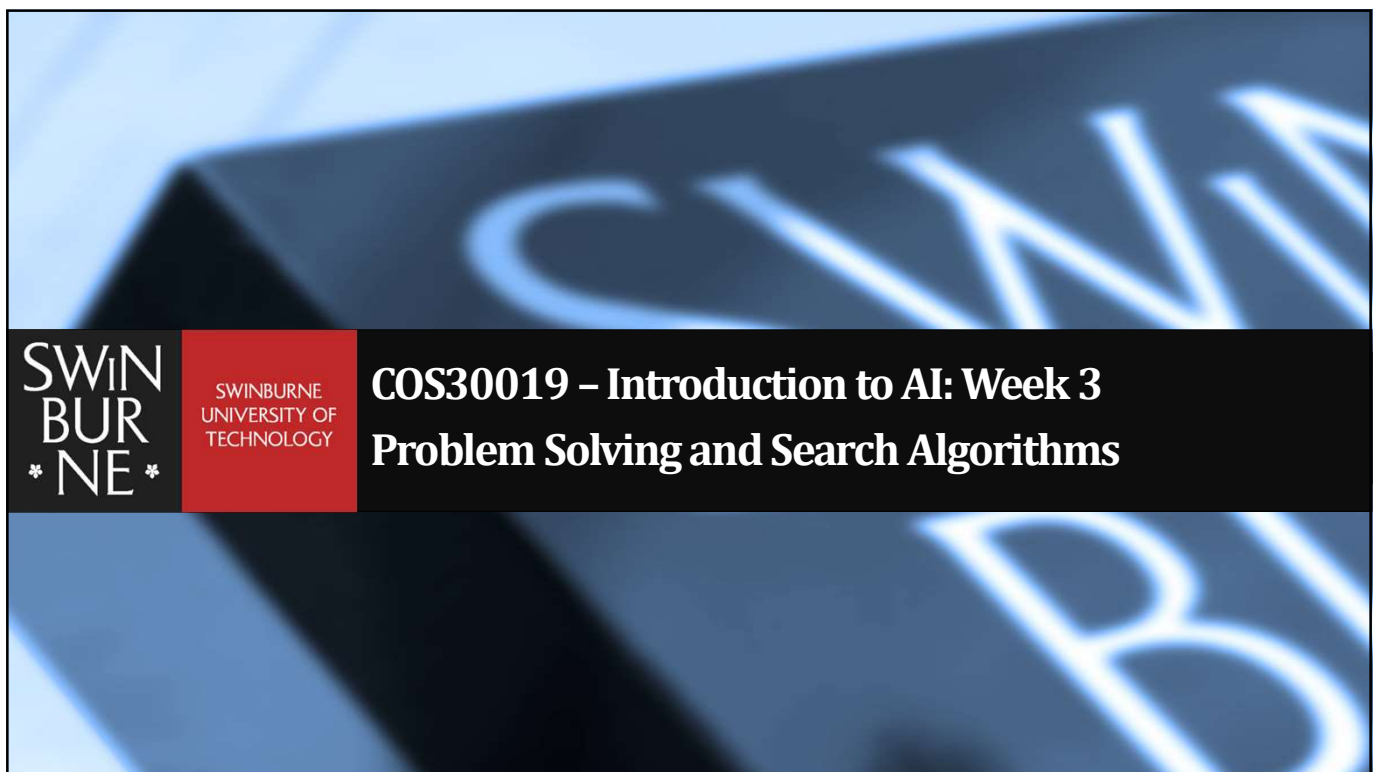




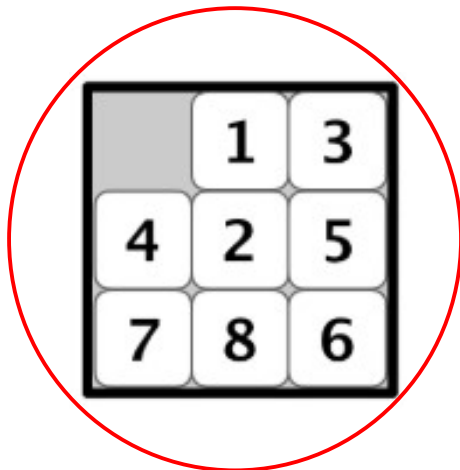
1

Overview

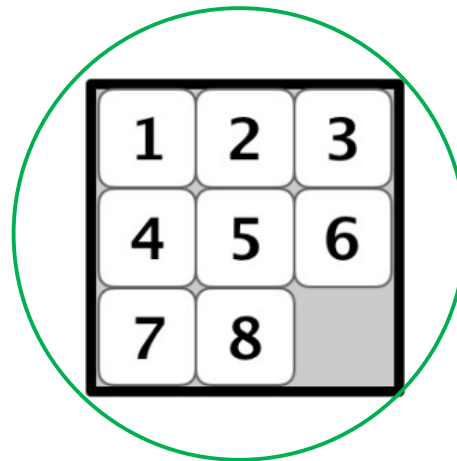
- Problem-solving agents
 - A kind of goal-based agent
- Problem types
 - Single state (fully observable)
- Problem formulation
 - Example problems
- Basic search algorithms
 - Uninformed

2

Example: 8-puzzle

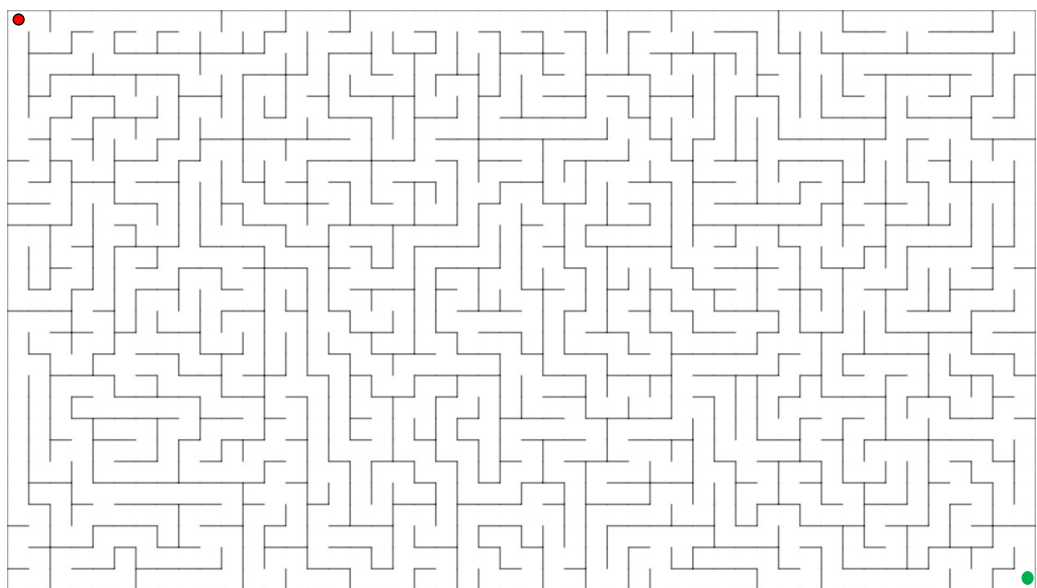


Initial State
(aka. Start State)

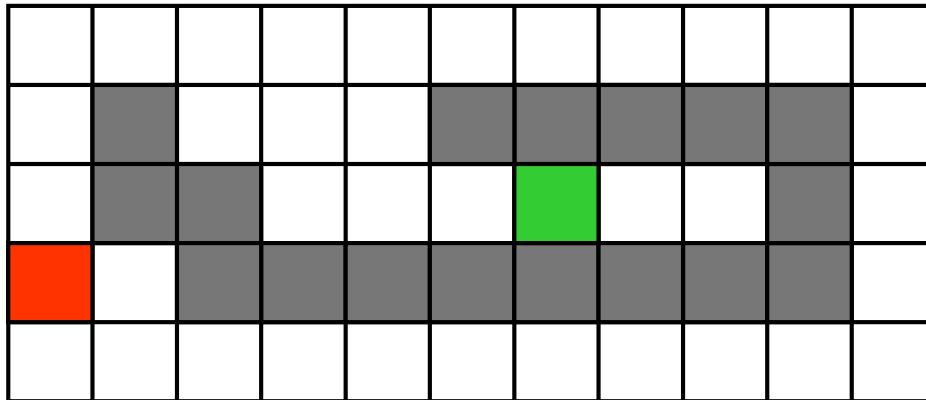


Goal State

Example: Solving a maze



Example: Robot Navigation

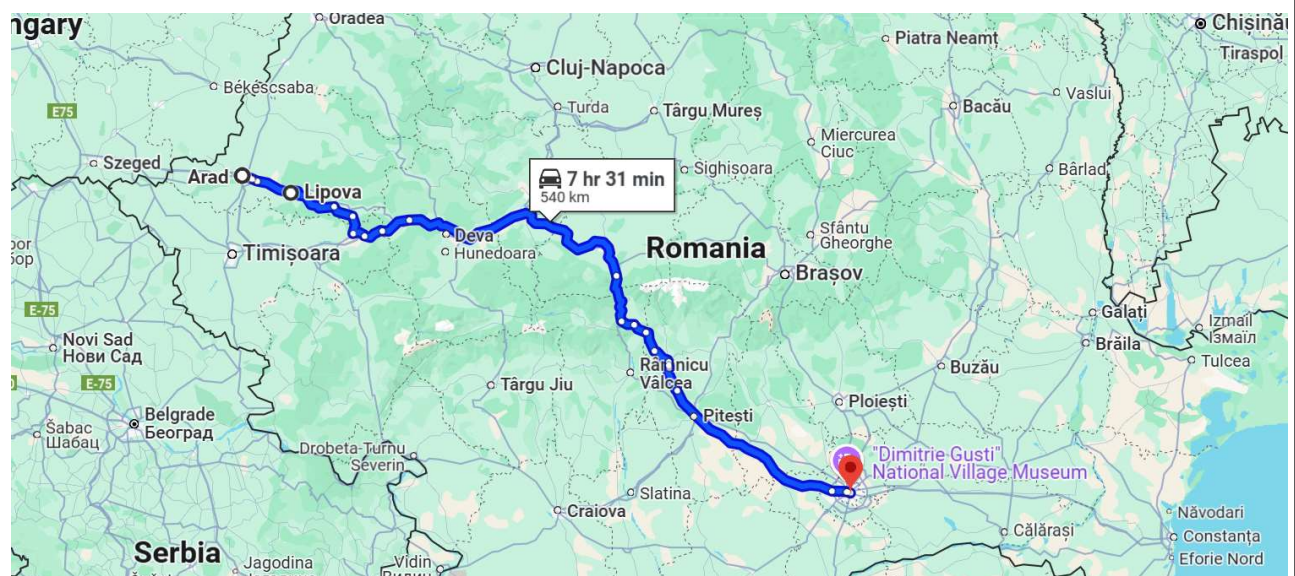


Initial state



Goal state

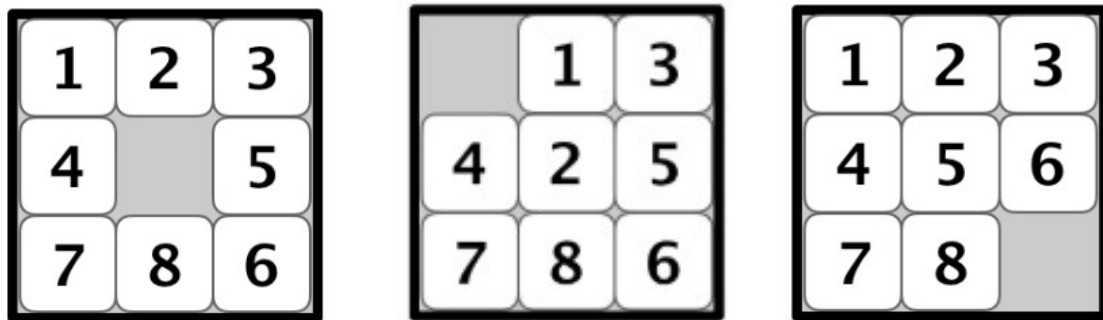
Example: Route finding



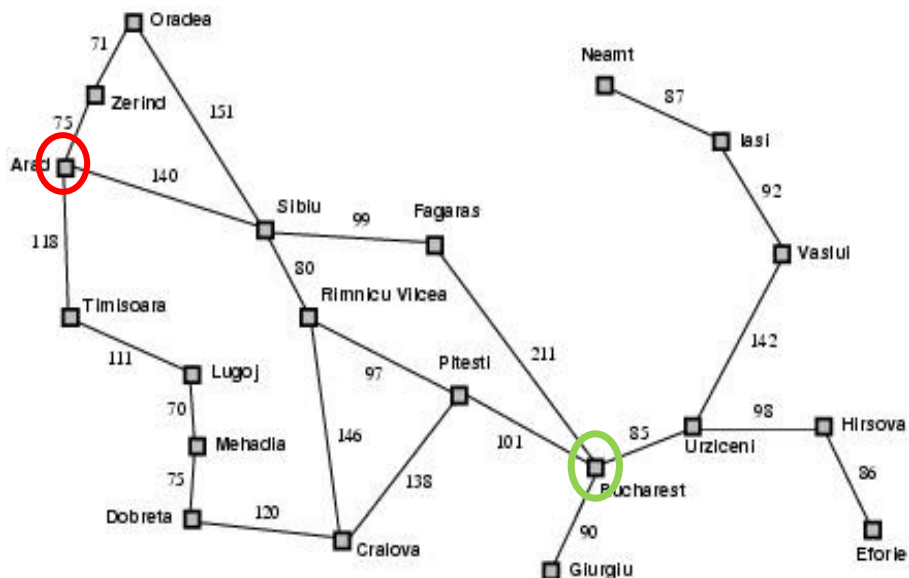
State



- A configuration of the agent and its environment



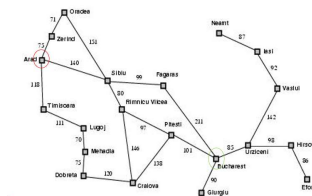
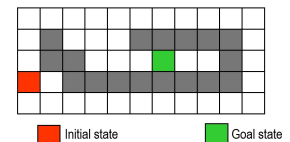
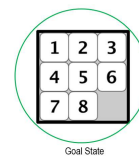
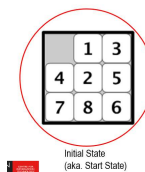
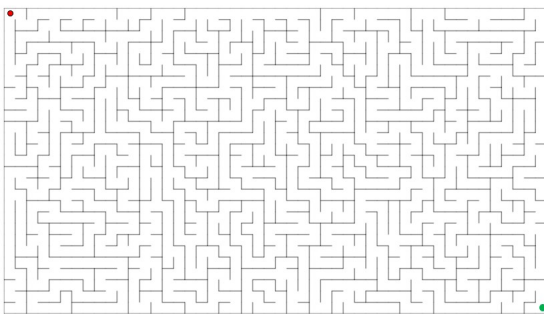
Example: Route finding





Initial state

- A state in which the agent begins
- i.e., the agent solves the problem by finding the sequence of **actions** that leads it from the *initial state* to a goal state



Actions



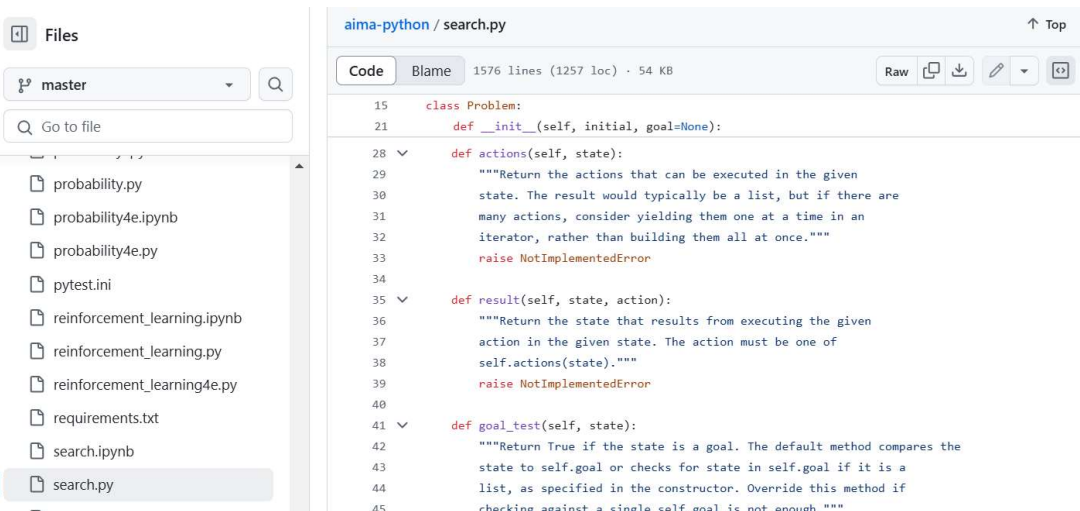
- Choices that can be made in a state (because the agent can perform them); aka. *decisions*.
 - Given a state s , $Actions(s)$ returns a set of actions/choices the agent can take in state s

Actions



- Choices that can be made in a state (because the agent can perform

- Give the agent



11

Transition model



- A description of the resulting state when executing an action a in a state s .
 - Given a state s and an applicable action a , $Result(s, a)$ returns *the state* resulting from executing a in state s .

12



Transition model

• A description of the resulting state when executing an action a in a state

• Give
return

Files

master

Go to file

- probability.py
- probability4e.ipynb
- probability4e.py
- pytest.ini
- reinforcement_learning.ipynb
- reinforcement_learning.py
- reinforcement_learning4e.py
- requirements.txt
- search.ipynb
- search.py

aima-python / search.py

Code Blame 1576 lines (1257 loc) · 54 KB

```

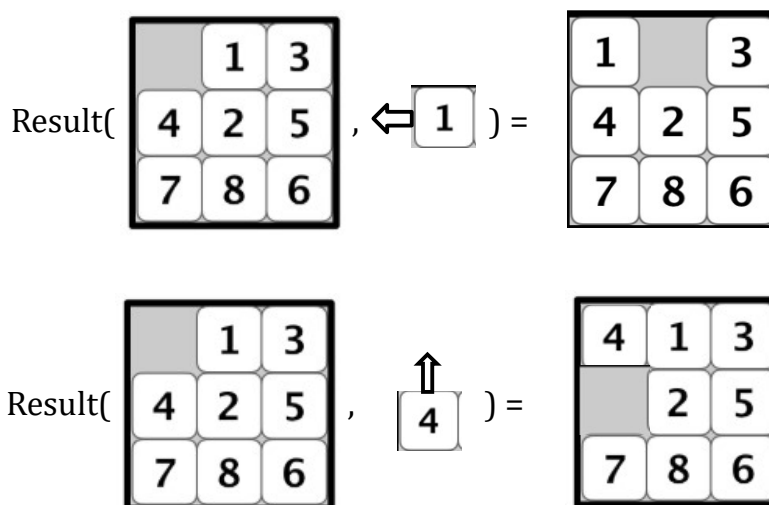
15 class Problem:
21     def __init__(self, initial, goal=None):
28     def actions(self, state):
29         """Return the actions that can be executed in the given
30         state. The result would typically be a list, but if there are
31         many actions, consider yielding them one at a time in an
32         iterator, rather than building them all at once."""
33         raise NotImplementedError
34
35     def result(self, state, action):
36         """Return the state that results from executing the given
37         action in the given state. The action must be one of
38         self.actions(state)."""
39         raise NotImplementedError
40
41     def goal_test(self, state):
42         """Return True if the state is a goal. The default method compares the
43         state to self.goal or checks for state in self.goal if it is a
44         list, as specified in the constructor. Override this method if
45         checking against a single self.goal is not enough."""

```

13



Transition model - Example

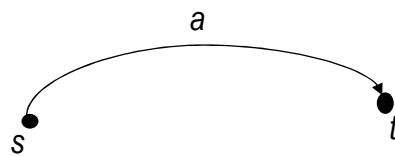


14

Transition model



- That is, if state $t = \text{Result}(s, a)$, we can transition (or, “go”) from state s to state t via action a ; visualised as:



State space & goal test



- *State space*:
 - The set of all states reachable from the initial state by any sequence of actions
- *Goal test*:
 - Way to determine whether a given state is a goal state



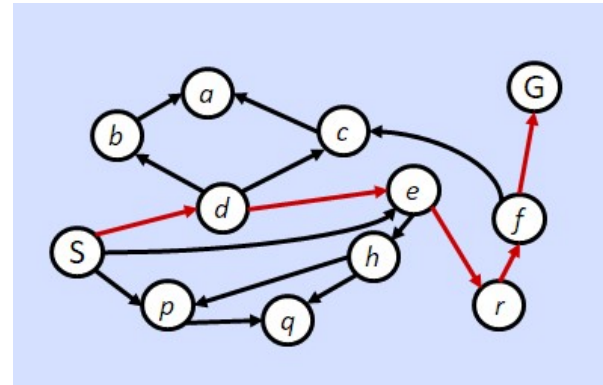
State space & goal test

■ State space:

- The set of all states reachable from the initial state by any sequence of actions

■ Goal test:

- Way to determine whether a given state is a goal state



Goal test = *Agent is in state G*



State space & goal test

■ State space:

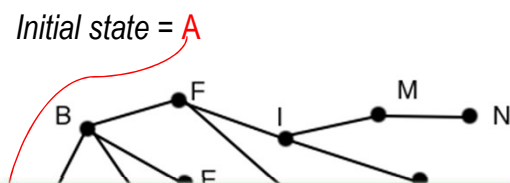
- The set of all states reachable from the initial state by any sequence of actions

■ Goal test:

- Way to determine whether a given state is a goal state

• But perhaps:

- **In C:** Agent having \$10M and being wanted by the police for bank robbery
- **In P:** Agent having \$10M after many years of hard work and saving and investing



Goal test = *Agent has \$10M*



Problem-solving agent

- Four general steps in problem solving:
 - ☐ Goal formulation
 - ☐ What are the successful world states → Goal test?
 - ☐ Problem formulation
 - ☐ What actions and states to consider given the goal → State space?
 - ☐ Search
 - ☐ Determine the possible sequence of actions that lead to the states of known values and then choosing the best sequence. (i.e., solution = the best sequence.)
 - ☐ Execute
 - ☐ Given the solution, perform the actions.



Problem-solving agent

- Four general steps in problem solving:
 - ☐ Goal formulation
 - ☐ What are the successful world states → Goal test?
 - ☐ Problem formulation
 - ☐ What actions and states to consider given the goal → State space?
 - ☐ Search
 - ☐ Determine the possible sequence of actions that lead to the states of known values and then choosing the best sequence. (i.e., solution = the best sequence.)
 - ☐ Execute
 - ☐ Given the solution, perform the actions.

Search problem (single-state)



- A **problem** is defined by the following:
 1. **initial state** e.g., "at Arad"
 2. **actions**
 3. **transition model** (i.e., the $Result(s,a)$ function)
 4. **goal test**; can be
 - ☐ **explicit**, e.g., $x = \text{"at Bucharest"}$
 - ☐ **implicit**, e.g., $Checkmate(x)$
 5. **path cost**:
 - ☐ Typically additive; e.g., sum of distances, number of actions executed, etc.
 - ☐ $c(s, a, t)$ is the **step cost** (for the step of taking action a to transition from state s to state t ; assumed to be ≥ 0)

A **solution** is a sequence of actions leading from the initial state to a **goal state**



21

Search problem (**single-state**)



- A **problem** is defined by the following:
 1. **initial state** e.g., "at Arad"
 2. **actions**
 3. **transition model**
 - A description of the resulting state when executing an action a in a state s .
 - Given a state s and an applicable action a , $Result(s, a)$ returns **the state** resulting from executing a in state s .
 4. **goal test**; can be
 - ☐ **explicit**, e.g., $x = \text{"at Bucharest"}$
 - ☐ **implicit**, e.g., $Checkmate(x)$
 5. **path cost**:
 - ☐ Typically additive; e.g., sum of distances, number of actions executed, etc.
 - ☐ $c(s, a, t)$ is the **step cost** (for the step of taking action a to transition from state s to state t ; assumed to be ≥ 0)

A **solution** is a sequence of actions leading from the initial state to a **goal state**

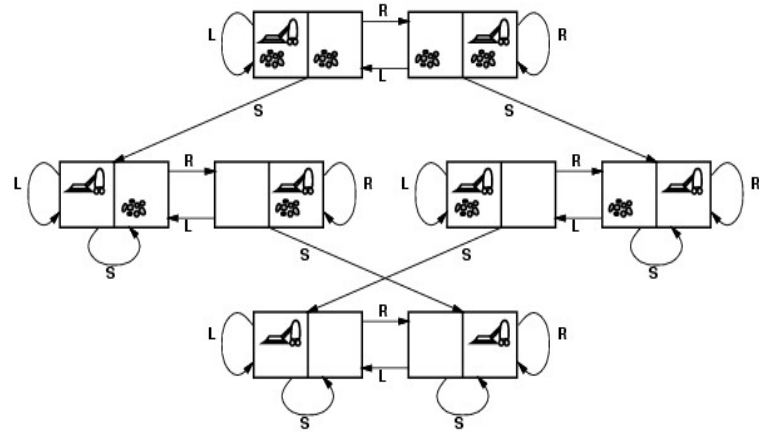


22



Vacuum world search problem

- states? integer dirt and robot location
- actions? Left, Right, Suck



- goal test? no dirt at all locations
- path cost? 1 per action



Problem types

- Deterministic, fully observable $\Rightarrow$ *single state problem*
 - Agent knows exactly which state it will be in; solution is a sequence.
- Partial knowledge of states and actions:
 - Non-observable $\Rightarrow$ *sensorless or conformant problem*
 - Agent may have no idea where it is; solution (if any) is a sequence.
 - Nondeterministic and/or partially observable $\Rightarrow$ *contingency problem*
 - Percepts provide *new* information about current state; solution is a tree or policy; often interleave search and execution.
 - Unknown state space $\Rightarrow$ *exploration problem* ("online")
 - When states and actions of the environment are unknown.

Solving problems using search algorithms

25

Basic search algorithms – An overview



- How do we find the solutions of previous problems?
 - Search the **state space** (remember complexity of space depends on state representation)
 - Here: search through *explicit tree generation*
 - ROOT= initial state.
 - Nodes are generated through **transition model** (i.e., the *Result(s, a)* function)
 - In general search generates a graph (same state through multiple paths)

26



Basic search algorithms – An overview

- How do we find the solutions of previous problems?
 - Search the **state space** (remember complexity of space depends on state representation)
 - Here: search through *explicit tree generation*
 - ROOT= initial state.
 - Nodes are generated through **transition model** (i.e., the *Result(s, a)* function)
 - In general
 - If this slide makes sense to you, you're good and can go having a cup of coffee and skip the next 10 minutes of this lecture.
 - If it does not, stay tuned.



The intuition behind search algorithms

- In any given state **s**:
 - By considering all actions applicable in **s**, we can see the available options from **s**
 - An option is a state reachable from **s** via one of the applicable actions
 - This step is called **exploring s** or sometimes **expanding s**.
 - Repeating the process by exploring those available options, we'll continue to see even more options
 - As more and more options are being discovered and some of them have been expanded while others are still available for expansion, we need a good data structure to organize them
 - The set of options that have not been expanded will be stored in a single data structure called **FRONTIER** to allow the search algorithm to consider them when it need to expand an option.



The intuition behind search algorithms

- In any given state s :
 - By considering all actions applicable in s , we can see the available options from s
 - An option is a state reachable from s via one of the applicable actions
 - This step is called **exploring** s or sometimes **expanding** s .
 - Repeating the process by exploring those available options, we'll continue to see even more options
 - As more and more options are explored while others remain unexplored, we need a way to organize them
 - The set of options that have not been expanded will be stored in a single data structure called **FRONTIER** to allow the search algorithm to consider them when it needs to expand an option.

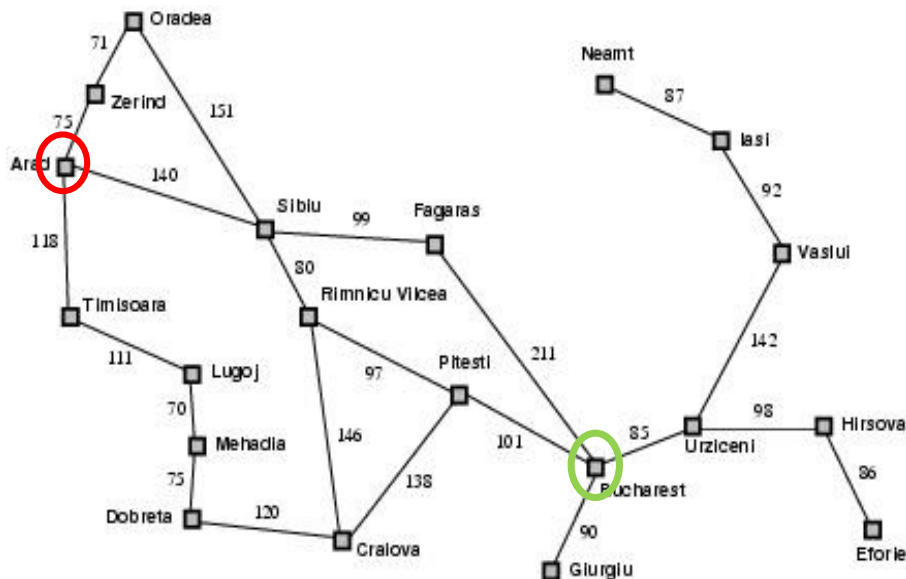
The **FRONTIER** represents all options that have not been explored before and that we can explore next.



Search algorithms - informally

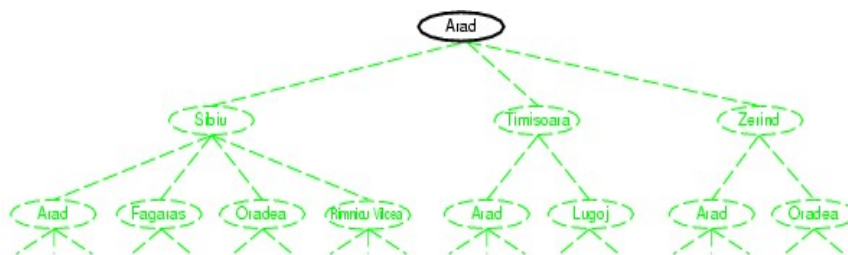
- Starts with a **FRONTIER** that contains the initial state
- Repeat:
 - If the **FRONTIER** happens to be empty, then **no solution**
 - Take a **node** out of the **FRONTIER** (this is *the option we choose to expand next*)
 - If that **node** contains a goal state, return the **found solution**
 - Else, expand **node**, add the resulting nodes (from **node**) to the **FRONTIER**

Example: Route finding



31

Simple tree search example



function TREE-SEARCH(*problem*, *frontier*) **return** a solution or failure

frontier ← INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *frontier*)

loop do

if EMPTY?(*frontier*) **then return** failure

node ← REMOVE-FIRST(*frontier*)

if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

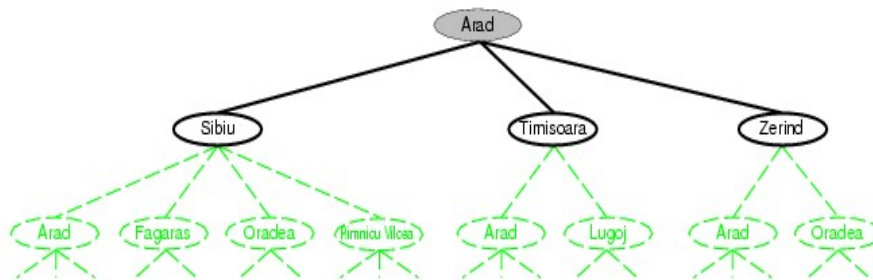
then return SOLUTION(*node*)

frontier ← INSERT-ALL(EXPAND(*node*, *problem*), *frontier*)

32



Simple tree search example



function TREE-SEARCH(*problem*, *frontier*) **return** a solution or failure

frontier $\leftarrow$ INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *frontier*)

loop do

if EMPTY?(*frontier*) **then return** failure

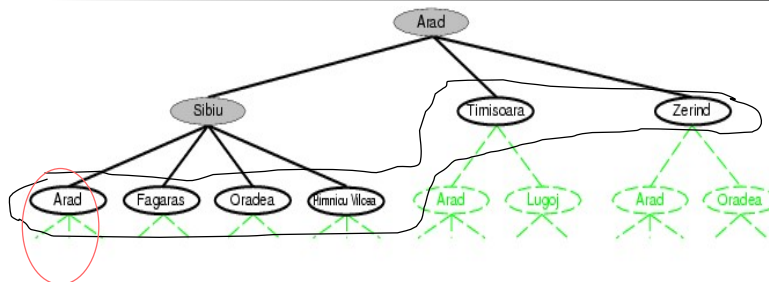
node $\leftarrow$ REMOVE-FIRST(*frontier*)

if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

then return SOLUTION(*node*)

frontier $\leftarrow$ INSERT-ALL(EXPAND(*node*, *problem*), *frontier*)

Simple tree search example



function TREE-SEARCH(*problem*, *frontier*) **return** a solution or failure

frontier $\leftarrow$ INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *frontier*)

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ REMOVE-FIRST(*frontier*)

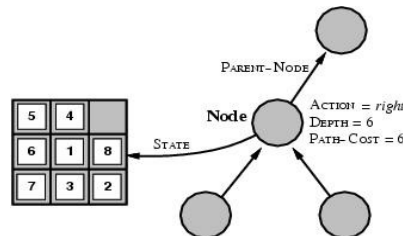
if GOAL-TEST[*problem*] applied to STATE[*node*] succeeds

then return SOLUTION(*node*)

frontier $\leftarrow$ INSERT-ALL(EXPAND(*node*, *problem*), *frontier*)

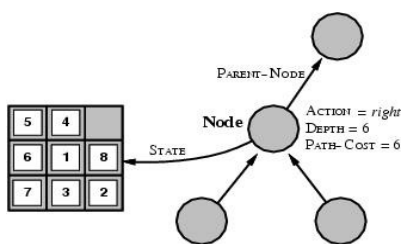
Determines search process!!

State space vs. search tree



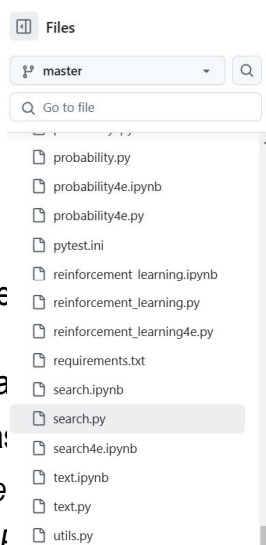
- A *state* is a (representation of) a physical configuration
- A *node* is a data structure belong to a search tree
 - A node has a parent, children, ... and includes path cost, depth, ...
 - Here *node* = $\langle \text{state}, \text{parent-node}, \text{action}, \text{path-cost}, \text{depth} \rangle$
 - *FRONTIER* = contains generated nodes which are not yet expanded.
 - White nodes with black outline

State space vs. search tree



- A *state* is a (re
- A *node* is a da
 - A node ha
 - Here *node*
 - *FRONTIE*

□ White nodes with black outline

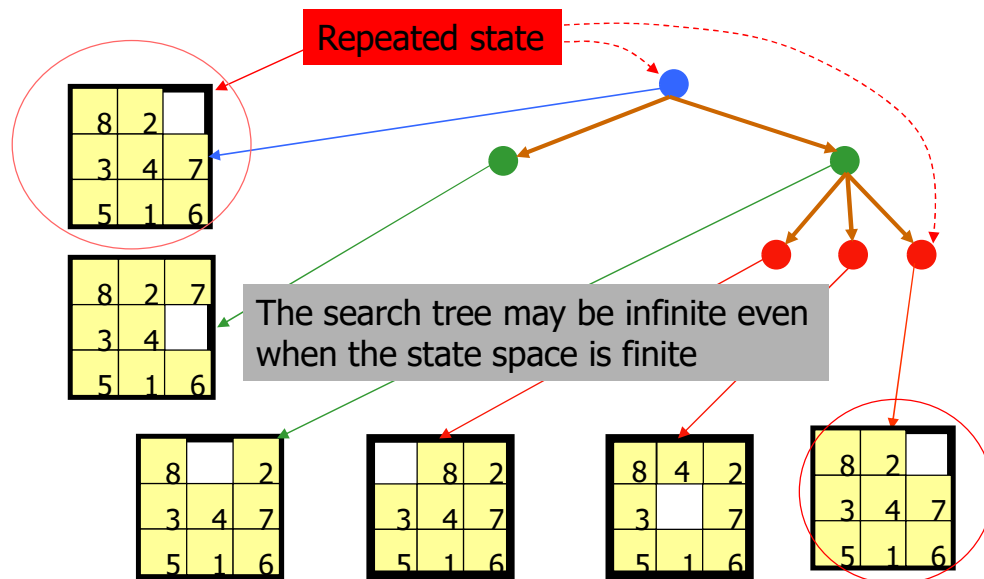


```

67
68 class Node:
69     """A node in a search tree. Contains a pointer to the parent (the node
70     that this is a successor of) and to the actual state for this node. Note
71     that if a state is arrived at by two paths, then there are two nodes with
72     the same state. Also includes the action that got us to this state, and
73     the total path_cost (also known as g) to reach the node. Other functions
74     may add an f and h value; see best_first_graph_search and astar_search for
75     an explanation of how the f and h values are handled. You will not need to
76     subclass this class."""
77
78     def __init__(self, state, parent=None, action=None, path_cost=0):
79         """Create a search tree Node, derived from a parent by an action."""
80         self.state = state
81         self.parent = parent
82         self.action = action
83         self.path_cost = path_cost
84         self.depth = 0
85         if parent:
86             self.depth = parent.depth + 1
87
88     def __repr__(self):
89         return "<Node {}>".format(self.state)
90
91     def __lt__(self, node):
92         return self.state < node.state
  
```

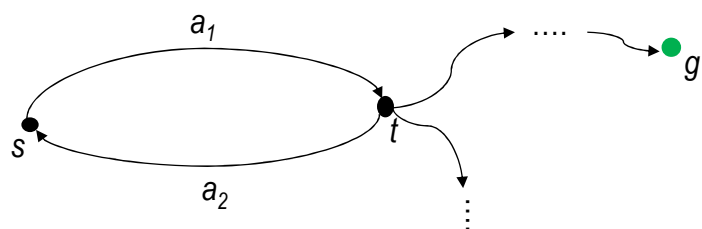


Search Nodes $\neq$ States



Issues with repeated states (aka. Duplicate states)

- At best, repeated states mean that the same search will be performed over and over again
 - $\rightarrow$ Waste of space & time
- At worst, it can lead to an infinite loop
 - For example, the search will be bounced back and forth between states s and t and will never get to a goal state





Search algorithms – Eliminate repeated states

- Starts with a **FRONTIER** that contains the initial state
- Initialise an empty **EXPLORED** set
- Repeat:
 - ❑ If the **FRONTIER** happens to be empty, then *no solution*
 - ❑ Take a *node* out of the **FRONTIER** (this is *the option we choose to expand next*)
 - ❑ If that *node* contains a goal state, return the *found solution*
 - ❑ Add *node.state* to the **EXPLORED** set
 - ❑ Expand *node*, add the resulting nodes (from *node*) to the **FRONTIER** if their states have not already been in the **FRONTIER** or the **EXPLORED** set



Search algorithms – Eliminate repeated states

- Starts with a **FRONTIER** that contains the initial state
 - Initialise an empty **EXPLORED** set
 - Repeat:
 - ❑ If the **FRONTIER** happens to be empty, then *no solution*
 - ❑ Take a *node* out of the **FRONTIER** (this is *the option we choose to expand next*)
 - ❑ If that *node* contains a goal state, return the *found solution*
 - ❑ Add *node.state* to the **EXPLORED** set
 - ❑ Expand *node*, add the resulting nodes (from *node*) to the **FRONTIER** if their states have not already been in the **FRONTIER** or the **EXPLORED** set
- Do pay attention to this statement because you may end up in a **suboptimal solution**.
WHY??

Search strategies



- A strategy is defined by picking the order of node expansion.
- Problem-solving performance is measured in four ways:
 - ☐ **Completeness**; *Does it always find a solution if one exists?*
 - ☐ **Optimality**; *Does it always find the least-cost solution?*
 - ☐ Time Complexity; *Number of nodes generated/expanded?*
 - ☐ Space Complexity; *Number of nodes stored in memory during search?*
- Time and space complexity are measured in terms of problem difficulty defined by:
 - ☐ b - maximum branching factor of the search tree
 - ☐ d - depth of the least-cost solution
 - ☐ m - maximum depth of the state space (may be ∞)

Blind vs. Heuristic Strategies



- **Blind** (or **uninformed**) strategies do not exploit any of the information contained in a state
- **Heuristic** (or **informed**) strategies exploits such information to assess that one node is “more promising” than another



Uninformed search strategies

■ Categories defined by expansion algorithm:

- ☒ Breadth-first search
- ☒ Uniform-cost search
- ☒ Depth-first search
- ☐ Depth-limited search
- ☐ Iterative deepening search.
- ☐ Bidirectional search



Uninformed search strategies

■ Breadth-first

- ☐ Bidirectional

■ Depth-first

- ☐ Depth-limited
- ☐ Iterative deepening

■ Uniform-Cost

■ (= Dijkstra)

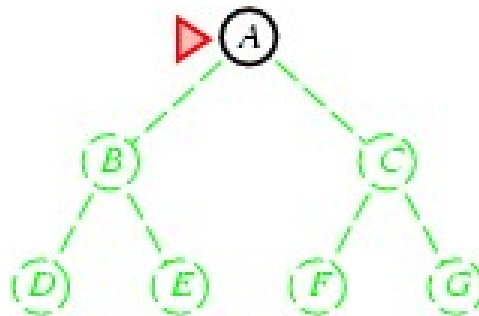
Step cost = 1

Step cost = $c(\text{action})$
 $\geq \epsilon > 0$



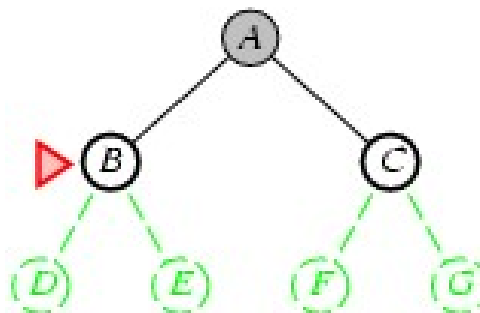
Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *frontier* is a FIFO queue, i.e., new successors go at end
 -



Breadth-first search

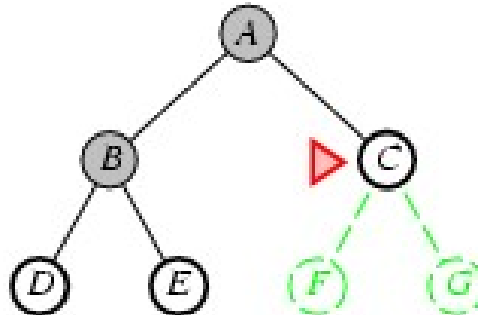
- Expand shallowest unexpanded node
- **Implementation:**
 - *frontier* is a FIFO queue, i.e., new successors go at end
 -





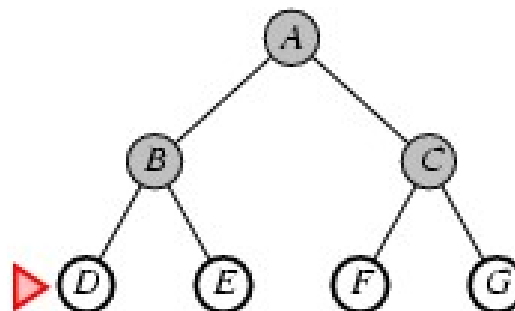
Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *frontier* is a FIFO queue, i.e., new successors go at end
 -



Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *frontier* is a FIFO queue, i.e., new successors go at end
 -



Breadth-first search



■ Expand

■ Implement

□ frontier



aima-python / search.py

Code Blame 1576 lines (1257 loc) · 54 KB

```

237
238 def breadth_first_graph_search(problem):
239     """[Figure 3.11]
240     Note that this function can be implemented in a
241     single line as below:
242     return graph_search(problem, FIFOQueue())
243     """
244     node = Node(problem.initial)
245     if problem.goal_test(node.state):
246         return node
247     frontier = deque([node])
248     explored = set()
249     while frontier:
250         node = frontier.popleft()
251         explored.add(node.state)
252         for child in node.expand(problem):
253             if child.state not in explored and child not in frontier:
254                 if problem.goal_test(child.state):
255                     return child
256             frontier.append(child)
257     return None
258
  
```

Files

master

Go to file

- probability.py
- probability4e.ipynb
- probability4e.py
- pytest.ini
- reinforcement_learning.ipynb
- reinforcement_learning.py
- reinforcement_learning4e.py
- requirements.txt
- search.ipynb
- search.py**
- search4e.ipynb
- text.ipynb

SWINBURNE CENTRE FOR INFORMATION TECHNOLOGY RESEARCH

49

Properties of breadth-first search



- Complete? Yes (if b is finite)
- Optimal? Yes (if cost = 1 per step)
-
- Time? $1+b+b^2+b^3+\dots +b^d + b(b^d-1) = O(b^{d+1})$
- Space? $O(b^{d+1})$ (keeps every node in memory)
- **Space** is the bigger problem (more than time)



50

Time and Memory Requirements



d	#Nodes	Time	Memory
2	111	.01 msec	11 Kbytes
4	11,111	1 msec	1 Mbyte
6	$\sim 10^6$	1 sec	100 Mb
8	$\sim 10^8$	100 sec	10 Gbytes
10	$\sim 10^{10}$	2.8 hours	1 Tbyte
12	$\sim 10^{12}$	11.6 days	100 Tbytes
14	$\sim 10^{14}$	3.2 years	10,000 Tb

Assumptions: $b = 10$; 1,000,000 nodes/sec; 100bytes/node

Time and Memory Requirements



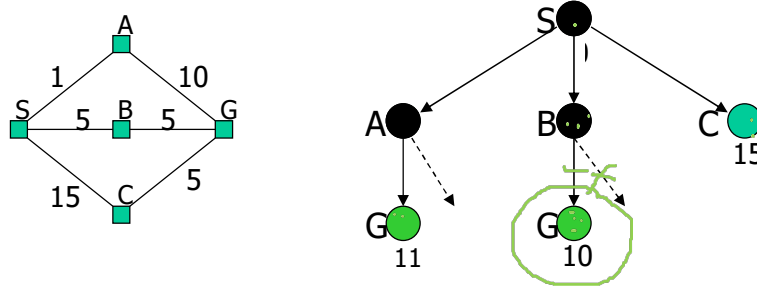
d	#Nodes	Time	Memory
2	111	.01 msec	11 Kbytes
4	11,111	1 msec	1 Mbyte
6	$\sim 10^6$	1 sec	100 Mb
8	$\sim 10^8$	100 sec	10 Gbytes
10	$\sim 10^{10}$	2.8 hours	1 Tbyte
12	$\sim 10^{12}$	11.6 days	100 Tbytes
14	$\sim 10^{14}$	3.2 years	10,000 Tb

Assumptions: $b = 10$; 1,000,000 nodes/sec; 100bytes/node

Uniform-Cost Strategy



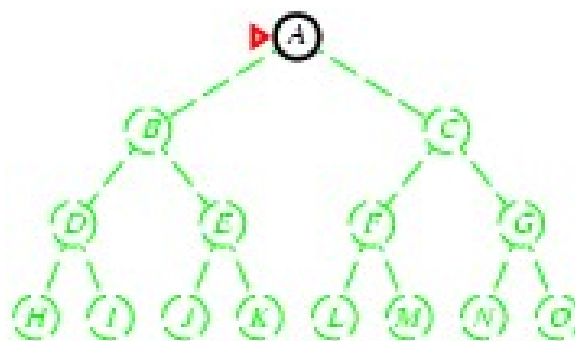
- Each step has some cost $\geq \epsilon > 0$.
- The cost of the path to each frontier node N is
 $g(N) = \sum \text{costs of all steps.}$
- The goal is to generate a solution path of minimal cost.
- The queue FRONTIER is sorted in increasing cost.



Depth-first search



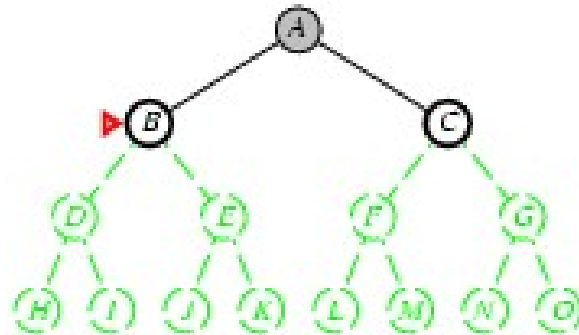
- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐





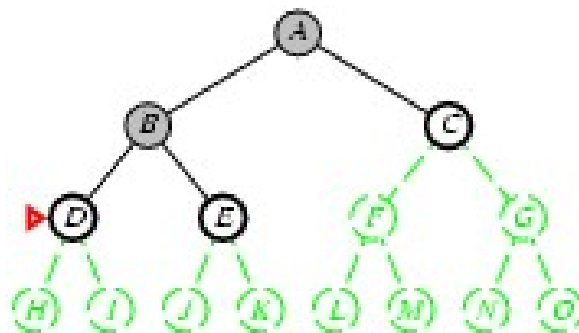
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐



Depth-first search

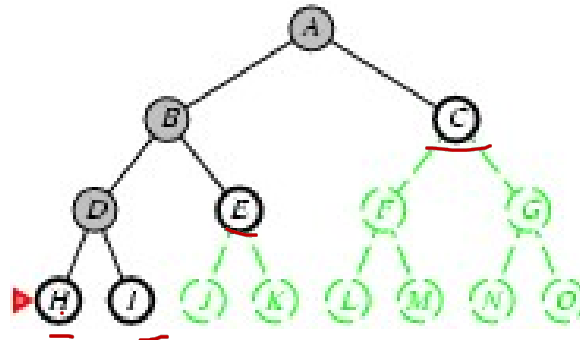
- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐





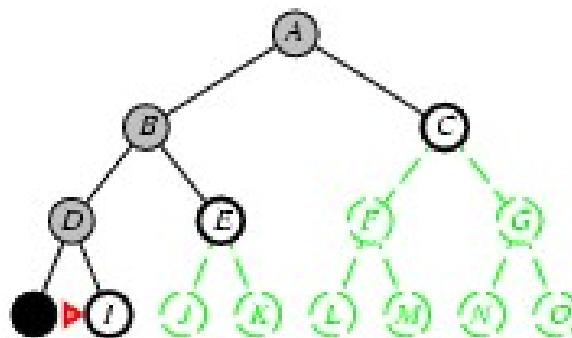
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐



Depth-first search

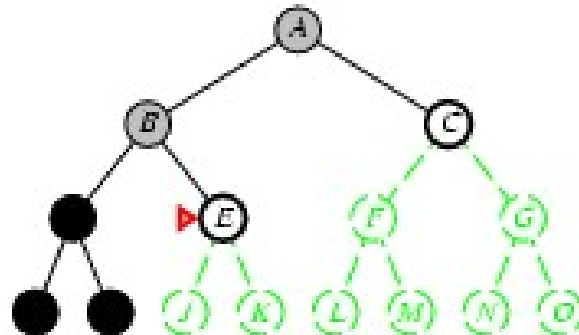
- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐





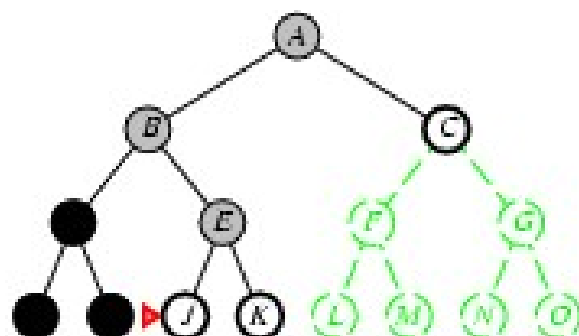
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐



Depth-first search

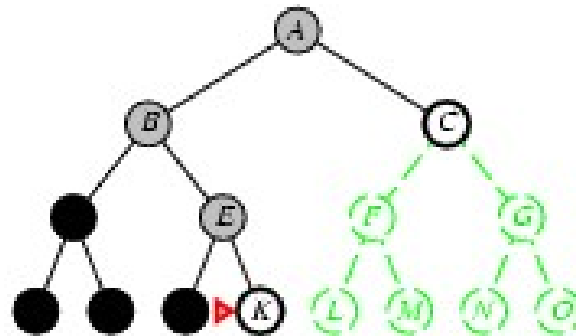
- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐





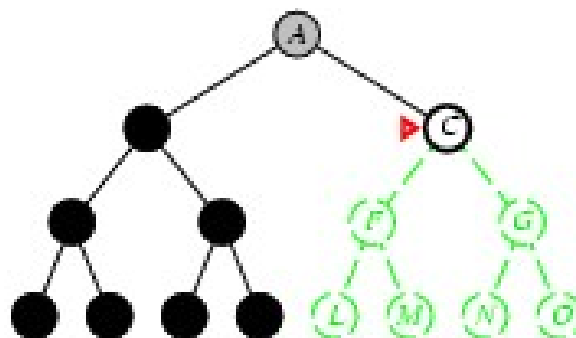
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐



Depth-first search

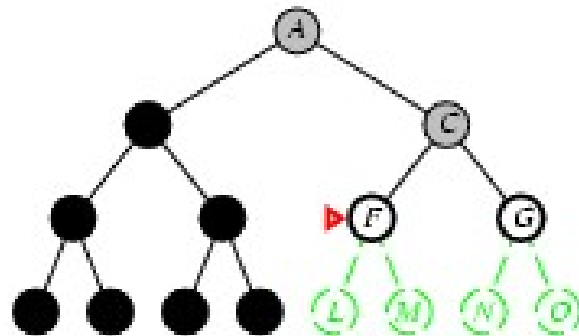
- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐





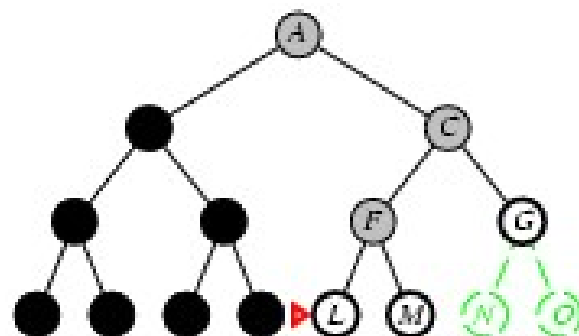
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐



Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - ☐ *frontier* = LIFO queue, i.e., put successors at front
 - ☐





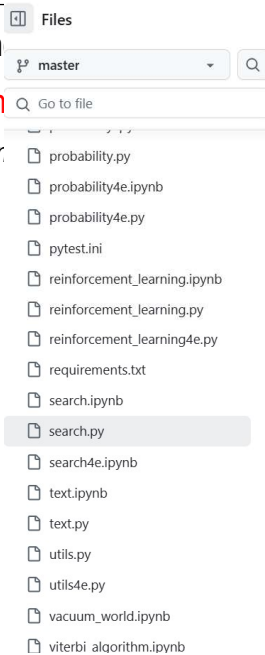
Depth-first search

■ Expan

■ Implem

□ for

□



```

196
197 def depth_first_tree_search(problem):
198     """
199     [Figure 3.7]
200     Search the deepest nodes in the search tree first.
201     Search through the successors of a problem to find a goal.
202     The argument frontier should be an empty queue.
203     Repeats infinitely in case of loops.
204     """
205
206     frontier = [Node(problem.initial)] # Stack
207
208     while frontier:
209         node = frontier.pop()
210         if problem.goal_test(node.state):
211             return node
212         frontier.extend(node.expand(problem))
213     return None
214
215
216 def depth_first_graph_search(problem):
217     """
218     [Figure 3.7]
219     Search the deepest nodes in the search tree first.
220     Search through the successors of a problem to find a goal.
221     The argument frontier should be an empty queue.
222     Does not get trapped by loops.
223     If two paths reach a state, only use the first one.
224     """
225     frontier = [(Node(problem.initial))] # Stack
  
```

65

Properties of depth-first search



■ Complete? No: fails in infinite-depth spaces, spaces with loops

□ Modify to avoid **repeated states** along path

→ complete in finite spaces

■ Optimal? No

■

■ Time? $O(b^m)$: terrible if m is much larger than d

□ but if solutions are dense, may be much faster than breadth-first

■ Space? $O(bm)$, i.e., linear space!

66



Summary

- Search tree $\neq$ state space
- Search strategies: breadth-first, depth-first, and variants
- Evaluation of strategies: completeness, optimality, time and space complexity
- Avoiding repeated states
- Optimal search with variable step costs